DELAVALOM STACK

Technical Design Document

The Definitive Claude Code Workflow System

Author: Luis Arvelo (Delavalom)

Version: 1.0.0

Date: March 28, 2026

Classification: Internal / Personal

Status: Active Development

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Sub-Agents Integration Design
4. Agent Teams Integration Design
5. The 13-Layer Architecture
6. Agent Definitions
7. One-Click Installation System
8. Plugin Manifest & Distribution
9. MCP Server Configuration
10. Hooks & Governance
11. Workflow Orchestration Patterns
12. Domain-Specific Workflows
13. Token Budget Analysis
14. Implementation Roadmap

1. Executive Summary

This document defines the complete technical architecture for the Delavalom Stack — a comprehensive Claude Code workflow system designed for Luis Arvelo's multi-domain development workflow spanning engineering, data infrastructure, marketing/GTM, ML/AI, and finance.

The stack consolidates 24+ repositories, 200+ skills, 76 CLAUDE.md files, and 24 MCP servers into a coherent, layered system with one-click installation. It integrates Claude Code's native sub-agents and experimental agent-teams features to create a 13-layer architecture that can handle everything from a quick bug fix to a full product sprint with parallel agent coordination.

Key Decisions

- Sub-agents are integrated as the primary delegation mechanism for single-session work — they preserve context, enforce tool restrictions, and route to optimal models.
- Agent teams are integrated as an optional 13th layer for complex parallel work requiring inter-agent communication (multi-file refactors, parallel reviews, competing hypothesis debugging).
- Installation is handled via a single shell script that installs the plugin, configures agents, sets up hooks, and initializes memory — zero manual setup required.
- The system is distributed as a Claude Code plugin with agents/, skills/, hooks, and MCP configs bundled together.

2. Architecture Overview

The Delavalom Stack is organized into 13 layers, each with a clear responsibility. Layers 1-12 operate within a single Claude Code session using sub-agents for delegation. Layer 13 activates agent teams for scenarios requiring parallel, communicating instances.

Layer	Name	Tool	Mode	Purpose
1	Identity	Master CLAUDE.md	Always	Style, rules, cross-project context
2	Orchestration	oh-my-claudecode	Always	Agent routing, task delegation
3	Methodology	superpowers	Always	TDD, debugging, review patterns
4	Quality	impeccable + no-use-effect	Always	Design + React enforcement
5	Sprint	gstack	Per-sprint	Plan → Build → Review → Ship
6	Marketing	marketingskills + Cowork	Per-domain	Content, SEO, CRO, email
7	Research	last30days + WebSearch	On-demand	Real-time market intelligence
8	Memory	auto-memory + claude-mem	Always	Short-term + long-term persistence
9	Data	bruin + RuVector	Per-domain	Pipelines, semantic search
10	Security	strix + gstack /cso	Pre-deploy	Pentesting, OWASP/STRIDE
11	Context Eng.	Agent-Skills-CE	Agent-building	Agent architecture patterns
12	Sub-Agents	Claude Code native	Always	7 custom sub-agents for delegation
13	Agent Teams	Claude Code experimental	On-demand	Parallel multi-instance work

3. Sub-Agents Integration Design

What Are Sub-Agents?

Sub-agents are specialized AI assistants that run in their own context window with a custom system prompt, specific tool access, and independent permissions. When Claude encounters a task that matches a sub-agent's description, it delegates to that sub-agent, which works independently and returns results. Sub-agents cannot spawn other sub-agents.

Why Sub-Agents for Delavalom?

- Context preservation — exploration and implementation stay out of the main conversation, preventing context bloat across your 24+ projects.
- Tool enforcement — a code reviewer cannot accidentally edit files; a researcher cannot push to git.
- Cost optimization — route simple tasks to Haiku (fast, cheap) while keeping Opus for complex reasoning.
- Reusability — user-level sub-agents work across ALL your projects from ~/.claude/agents/.
- Progressive specialization — each sub-agent has a focused system prompt for its domain.

Sub-Agent Configuration

Sub-agents are defined as Markdown files with YAML frontmatter. They support the following fields:

Field	Required	Description
name	Yes	Unique identifier (lowercase + hyphens)
description	Yes	When Claude should delegate to this sub-agent
tools	No	Allowlist of tools. Inherits all if omitted
disallowedTools	No	Denylist — removed from inherited tools
model	No	sonnet, opus, haiku, full ID, or inherit
permissionMode	No	default, acceptEdits, dontAsk, bypassPermissions, plan
maxTurns	No	Maximum agentic turns before stopping
skills	No	Skills to inject into context at startup
mcpServers	No	MCP servers available to this sub-agent
hooks	No	Lifecycle hooks scoped to this sub-agent
memory	No	Persistent memory scope: user, project, or local
background	No	Run as background task (true/false)
isolation	No	Set to "worktree" for git worktree isolation

effort	No	low, medium, high, or max (Opus only)

Built-in Sub-Agents

Claude Code includes 3 built-in sub-agents:

- Explore — Haiku model, read-only, fast codebase search and analysis.
- Plan — Inherits model, read-only, gathers context for plan mode.
- General-purpose — Inherits model, all tools, complex multi-step tasks.

Custom Delavalom Sub-Agents (7 agents)

The following 7 custom sub-agents are designed for Luis's specific workflow domains. They are stored in .claude/agents/ within the stack and distributed via the plugin.

Agent	Model	Tools	Purpose
delavalom-reviewer	sonnet	Read, Grep, Glob, Bash	Staff-level code review with security, performance, and architecture analysis. Use proactively after any code change.
delavalom-debugger	inherit	Read, Edit, Bash, Grep, Glob	Root-cause analysis specialist. Captures errors, forms hypotheses, implements minimal fixes, verifies solutions.
delavalom-researcher	haiku	Read, Grep, Glob, WebSearch, WebFetch	Fast research agent for market trends, competitor analysis, documentation lookup. Cost-optimized with Haiku.
delavalom-k8s-ops	sonnet	Bash, Read, Grep	Kubernetes operations — pod debugging, deployment strategies, Helm charts, multi-cell management.
delavalom-data-eng	inherit	Bash, Read, Write, Edit	Data pipeline specialist — bruin pipelines, SQL transforms, data quality checks, RuVector integrations.
delavalom-frontend	sonnet	Read, Edit, Write, Bash, Grep, Glob	Frontend specialist with impeccable design skills and no-use-effect React enforcement loaded.
delavalom-content	sonnet	Read, Write, WebSearch, WebFetch	Content creation and distribution — generates platform-specific versions following the distribute pipeline.

Sub-Agent Scoping Strategy

Sub-agents are stored at two levels for maximum flexibility:

- User-level (~/.claude/agents/) — delavalom-reviewer, delavalom-debugger, delavalom-researcher. These work across ALL projects.
- Project-level (.claude/agents/) — delavalom-k8s-ops, delavalom-data-eng, delavalom-frontend, delavalom-content. These are specific to the Delavalom stack.

Sub-Agent Memory Configuration

Each sub-agent that benefits from learning across sessions is configured with persistent memory:

- delavalom-reviewer: memory: user — learns coding patterns across all projects
- delavalom-debugger: memory: project — learns project-specific failure patterns
- delavalom-frontend: memory: user — accumulates design system knowledge
- delavalom-content: memory: user — remembers brand voice and audience insights

4. Agent Teams Integration Design

What Are Agent Teams?

Agent teams coordinate multiple Claude Code instances working together. One session acts as the team lead, coordinating work, assigning tasks, and synthesizing results. Teammates work independently, each in its own context window, and communicate directly with each other. Unlike sub-agents, teammates can share findings and challenge each other.

Agent teams are experimental (CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1) and require Claude Code v2.1.32+. They use significantly more tokens than a single session but excel at genuinely parallel work.

When to Use Agent Teams vs Sub-Agents

Dimension	Sub-Agents	Agent Teams
Context	Own window, results return to caller	Own window, fully independent
Communication	Report back to main agent only	Teammates message each other directly
Coordination	Main agent manages all work	Shared task list, self-coordination
Best for	Focused tasks where only result matters	Complex work requiring discussion
Token cost	Lower (results summarized)	Higher (each is separate instance)
When to use	Daily tasks, reviews, research	Multi-file refactors, parallel debugging

Agent Teams Use Cases for Delavalom

- Parallel code review — 3 teammates review the same PR from security, performance, and test coverage angles simultaneously.
- Competing hypothesis debugging — 5 teammates investigate different root causes in parallel, debating each other's theories.
- Cross-layer implementation — one teammate handles frontend (TypeScript), another handles backend (Go/Python), another writes tests.
- Multi-project refactor — teammates each own a different project's migration while coordinating shared interfaces.
- Research sprints — teammates research different market segments, competitors, or technology options simultaneously.

Agent Teams Configuration

Enable agent teams by adding to .claude/settings.json:

```
{ "env": { "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1" } }
```

How It Fits Into Layer 13

Agent teams are the 13th layer — activated on-demand when a task genuinely benefits from parallel, communicating instances. The decision tree is simple:

- Single focused task? → Use a sub-agent (Layer 12)
- Multiple independent tasks? → Use parallel sub-agents (Layer 12)
- Tasks need inter-agent communication? → Use an agent team (Layer 13)
- Sequential workflow? → Stay in main conversation with skills

Best Practices for Agent Teams

- Start with 3-5 teammates — more creates coordination overhead without proportional benefit.
- Size tasks for 5-6 per teammate — keeps everyone productive without excessive context switching.
- Give teammates enough context — they don't inherit the lead's conversation history.
- Avoid file conflicts — break work so each teammate owns different files.
- Pre-approve permissions — add common operations to permissions.allow before spawning.
- Use TeammateIdle hooks — enforce quality gates when teammates finish work.

5. The 13-Layer Architecture

The original report defined a 12-layer system. With the addition of native Claude Code sub-agents and agent teams, the architecture expands to 13 layers organized into 4 tiers:

Tier A: Always-Active Foundation (Layers 1-4)

- Layer 1 — Identity: Master CLAUDE.md with Luis's style, rules, language preferences, project registry.
- Layer 2 — Orchestration: oh-my-claudecode for agent routing, smart model selection, token optimization.
- Layer 3 — Methodology: superpowers for TDD (RED-GREEN-REFACTOR), systematic debugging, code review loops.
- Layer 4 — Quality: impeccable for frontend design enforcement, no-use-effect for React anti-patterns.

Tier B: Domain-Activated Layers (Layers 5-9)

- Layer 5 — Sprint: gstack workflow (/review, /qa, /ship, /cso, /canary, /retro).
- Layer 6 — Marketing: marketingskills (34 skills) + Cowork marketing plugin (8 skills).
- Layer 7 — Research: last30days-skill for real-time aggregation across 10+ platforms.
- Layer 8 — Memory: Cowork auto-memory for preferences, claude-mem for project-level persistence.
- Layer 9 — Data: bruin pipelines for ETL, RuVector for semantic search and knowledge graphs.

Tier C: Governance & Specialization (Layers 10-12)

- Layer 10 — Security: strix autonomous pentesting + gstack /cso for OWASP/STRIDE quick checks.
- Layer 11 — Context Engineering: Agent-Skills-CE for building new agents and skills.
- Layer 12 — Sub-Agents: 7 custom sub-agents for delegation (reviewer, debugger, researcher, k8s-ops, data-eng, frontend, content).

Tier D: Parallel Coordination (Layer 13)

- Layer 13 — Agent Teams: Experimental multi-instance coordination for tasks requiring parallel work with inter-agent communication.

6. Agent Definitions

Seven custom sub-agents are defined as Markdown files with YAML frontmatter. Each is stored in .claude/agents/ and distributed via the Delavalom plugin.

Agent: delavalom-reviewer

Model: sonnet

Tools: Read, Grep, Glob, Bash

Skills: superpowers

Memory: user

Purpose: Staff-level code review

System Prompt: You are a senior staff engineer performing code review. Run git diff, focus on modified files, and provide feedback organized by priority: Critical (must fix), Warnings (should fix), Suggestions (consider improving). Check for security issues, performance problems, and DRY violations. Include specific examples of how to fix each issue.

Agent: delavalom-debugger

Model: inherit

Tools: Read, Edit, Bash, Grep, Glob

Skills: superpowers

Memory: project

Purpose: Root-cause analysis

System Prompt: You are an expert debugger. Capture the error, identify reproduction steps, isolate the failure, implement a minimal fix, and verify the solution. Focus on the underlying cause, not symptoms. Add strategic debug logging. Provide prevention recommendations.

Agent: delavalom-researcher

Model: haiku

Tools: Read, Grep, Glob

Purpose: Fast research and discovery

System Prompt: You are a fast research agent. Search codebases, documentation, and files to answer questions. Return concise, actionable summaries. Cite specific files and line numbers.

Agent: delavalom-k8s-ops

Model: sonnet

Tools: Bash, Read, Grep

Skills: delavalom-k8s

Purpose: Kubernetes operations

System Prompt: You are a Kubernetes operations specialist. Debug pod failures, review Helm charts, implement deployment strategies, and manage multi-cell architectures. Follow the debugging playbook: describe pod, check logs, verify resources, test networking.

Agent: delavalom-data-eng

Model: inherit

Tools: Bash, Read, Write, Edit

Memory: project

Purpose: Data pipeline work

System Prompt: You are a data engineering specialist. Build bruin pipelines, write SQL/Python transforms, implement data quality checks, and integrate with RuVector for semantic search. Follow DRY principles for pipeline code.

Agent: delavalom-frontend

Model: sonnet

Tools: Read, Edit, Write, Bash, Grep, Glob

Skills: impeccable, no-use-effect

Memory: user

Purpose: Frontend development

System Prompt: You are a frontend specialist. Write TypeScript + React code following impeccable design principles. Enforce no-use-effect patterns. Use Tailwind for styling. Run /audit before completing any UI work.

Agent: delavalom-content

Model: sonnet

Tools: Read, Write

Skills: delavalom-distribute

Memory: user

Purpose: Content creation and distribution

System Prompt: You are a content creation and distribution specialist. Generate platform-optimized versions of content following the distribution matrix. Adapt tone, length, and CTA for each platform. Apply SEO best practices for blog content.

7. One-Click Installation System

The Delavalom Stack is installed with a single command that sets up agents, skills, hooks, MCP configs, and memory. The installer supports three modes:

Mode	Command	What It Does
Full Install	bash install.sh --full	Agents + skills + hooks + MCP + memory + settings
Minimal	bash install.sh --minimal	Agents + CLAUDE.md only (fastest)
Plugin	claude plugins install delavalom-stack	Install from plugin marketplace

Installation Flow

- Step 1: Detect existing Claude Code installation and version.
- Step 2: Copy agent definitions to ~/.claude/agents/ (user-level) and .claude/agents/ (project-level).
- Step 3: Copy custom skills to .claude/skills/.
- Step 4: Merge hooks into .claude/settings.json (preserving existing hooks).
- Step 5: Merge MCP server configs into .claude/mcp.json.
- Step 6: Enable agent teams experimental flag in settings.json.
- Step 7: Initialize memory system with user profile.
- Step 8: Run claude agents to verify installation.

Uninstallation

bash install.sh --uninstall removes all Delavalom-specific files without affecting other Claude Code configurations. It preserves user data in memory directories.

8. Plugin Manifest & Distribution

The Delavalom Stack is packaged as a Claude Code plugin for distribution via the plugin marketplace or direct installation from the repository.

Plugin Structure

```
delavalom-stack/.claude-plugin/
├── plugin.json          (Metadata, version, dependencies)
├── README.md            (Installation instructions)
├── hooks/hooks.json     (Lifecycle hooks)
└── scripts/install.sh   (Post-install setup)
```

Distribution Channels

- Claude Code Plugin Marketplace — publish via claude plugins publish for one-click install by anyone.
- GitHub Repository — clone and install locally with bash install.sh.
- NPM Package — optional npm distribution for CI/CD integration.

9. MCP Server Configuration

The stack configures 7 curated MCP servers (down from 24 found across repos) to minimize overhead:

Server	Category	Purpose
github	Essential	PR/issue/repo operations
filesystem	Essential	Local file operations
memory	Essential	Persistent memory across sessions
sequential-thinking	Essential	Chain-of-thought for complex problems
playwright	Development	Browser automation and testing
context7	Development	Live documentation lookup
exa-web-search	Research	Deep web search via Exa API

10. Hooks & Governance

Hook Event	Matcher	Action	Purpose
PreToolUse	Bash	Block --no-verify	Prevent bypassing git hooks
PreToolUse	Bash	git push reminder	Remind to /review and /cso first
PreToolUse	Write	Doc file warning	Ensure doc consistency with CLAUDE.md
SubagentStart	delavalom-*	Log agent start	Track which agents are activated
TeammateIdle	*	Quality gate check	Verify work quality before idle

11. Workflow Orchestration Patterns

Pattern 1: Single-Session with Sub-Agent Delegation

The default workflow. Claude routes tasks to specialized sub-agents based on their descriptions. Results return to the main conversation for synthesis.

- User describes task → Claude matches sub-agent → Sub-agent works independently → Result returns
- Example: "Review my auth changes" → delavalom-reviewer runs git diff, analyzes, returns findings.

Pattern 2: Chained Sub-Agents

Multi-step workflows where each sub-agent's output feeds the next.

- Use delavalom-reviewer to find issues → delavalom-debugger to fix them → delavalom-reviewer to verify fixes.

Pattern 3: Parallel Sub-Agents

Independent tasks running simultaneously in separate context windows.

- Research auth module + database module + API module in parallel using 3 delavalom-researcher instances.

Pattern 4: Agent Team Sprint

Full parallel coordination for complex features spanning multiple domains.

- Lead creates team with frontend, backend, and test teammates.
- Each teammate owns their files, communicates via shared task list and messages.
- Lead synthesizes results and runs final integration review.

12. Domain-Specific Workflows

Engineering & Infrastructure

Active Agents: delavalom-reviewer, delavalom-debugger, delavalom-k8s-ops, delavalom-frontend

Flow: Start → OMC routes task → Sub-agent executes (TDD via superpowers) → delavalom-reviewer checks → gstack /cso security → gstack /ship deploys.

Data Engineering & ML

Active Agents: delavalom-data-eng, delavalom-researcher

Flow: Start → bruin pipeline design → delavalom-data-eng implements transforms → Data quality checks → RuVector indexing → API serving.

Marketing & Go-to-Market

Active Agents: delavalom-content, delavalom-researcher

Flow: Start → delavalom-researcher for trends → Cowork campaign-plan for strategy → delavalom-content generates platform versions → marketingskills seo-audit optimizes → Distribute.

Finance

Active Agents: delavalom-data-eng

Flow: Start → Receive files → PDF/XLSX skill extracts data → delavalom-data-eng processes → Generate reports.

13. Token Budget Analysis

Component	Estimated Tokens	When Loaded
Master CLAUDE.md	~3,000	Always
oh-my-claudecode context	~5,000	Always
superpowers skills	~4,000	Always
impeccable + no-use-effect	~3,000	Always (frontend)
Sub-agent definitions (7)	~2,000	Metadata only; full prompt on delegation
gstack skills	~4,000	Per-sprint
marketingskills	~3,000	Per-domain
Total always-active	~15,000-17,000	—
Total with domain layer	~22,000-28,000	—

This is well within Claude's optimal attention range. Sub-agents add minimal overhead (only the description is loaded into the main context; the full prompt loads when delegated). Agent teams have no impact on the main session's token budget since each teammate is a separate instance.

14. Implementation Roadmap

Phase	Deliverable	Timeline	Status
Phase 1	Interactive HTML research report	Day 1	Complete
Phase 1	Master CLAUDE.md	Day 1	Complete
Phase 1	4 custom skills (k8s, distribute, polyglot, cross-project)	Day 1	Complete
Phase 1	Hooks + MCP configuration	Day 1	Complete
Phase 1	Memory system initialization	Day 1	Complete
Phase 2	Technical Design Document	Day 1	Complete
Phase 2	7 custom sub-agent definitions	Day 1	Complete
Phase 2	One-click installer script	Day 1	Complete
Phase 2	Plugin manifest for distribution	Day 1	Complete
Phase 3	Agent team workflow templates	Week 2	Planned
Phase 3	Cross-project knowledge graph in RuVector	Week 3	Planned
Phase 4	Publish to Claude Code plugin marketplace	Week 4	Planned
Phase 4	Community feedback + iteration	Ongoing	Planned